

CLICK PLC Multi-Mode Light Demonstration

Main program with I/O, Forward, Reverse, and Random subroutines

Local pushbuttons plus FUXA HMI remote control over Modbus TCP, 0.75-second pacing, and six physical light outputs.

Design decision

The project is divided into five CLICK program units: **Main Program**, **I/O**, **Forward**, **Reverse**, and **Random**. Main owns mode arbitration, initialization, the shared pace timer, subroutine calls, and a single requested-light selector. Local pushbuttons and the PC HMI are first combined into three effective request bits. The HMI owns DS30: 0=Stop, 1=Forward, 2=Reverse, and 3=Random. The PLC reads DS30 but does not continuously write it. Each mode computes a requested light number in a mode-specific data register; Main selects one final request in DS20 and performs a simple one-of-six decode to C101--C106. Thus every command bit still has exactly one writer. The I/O subroutine is intentionally called at both the beginning and end of each scan: first to acquire inputs, then to apply the newly calculated output image.

1 Requirements and I/O

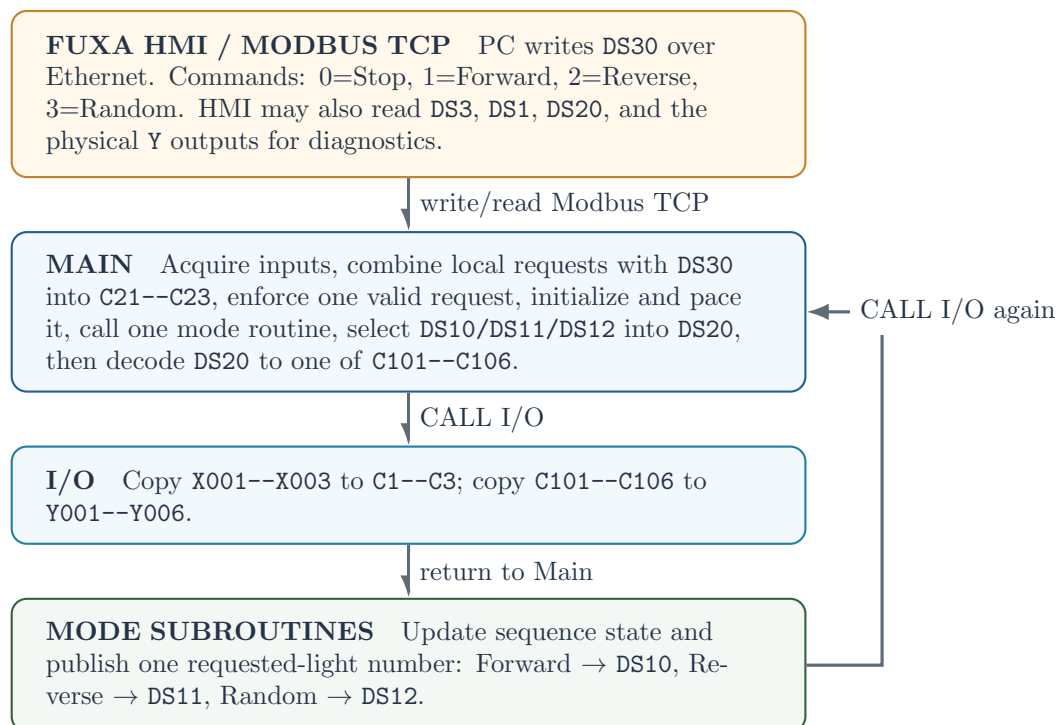
A mode may be requested either from the local pushbuttons or from the FUXA HMI. X001 requests Forward, X002 requests Reverse, and X003 requests Random. The HMI writes the same choices into DS30: 1=Forward, 2=Reverse, 3=Random, and 0=Stop/no HMI request. The PLC combines both sources and accepts a mode only when exactly one effective request is present. Conflicting requests fail off and all lights are commanded off.

Address	Assigned function	Address	Assigned function
X001	Forward request	Y001	Light 1
X002	Reverse request	Y002	Light 2
X003	Random request	Y003	Light 3
C1--C3	Input images	Y004	Light 4
C11--C13	Exclusive mode bits	Y005	Light 5
C14	Exactly-one-request valid	Y006	Light 6
C21--C23	Effective Forward/Reverse/Random requests	T1	Shared 0.75-second timer
C101--C106	Centralized output commands (single writer)	DS30	HMI mode request, 0 through 3
DS10	Forward requested light, 1 through 6	DS11	Reverse requested light, 1 through 6
DS12	Random requested light, 1 through 6	DS20	Final requested light, 0 through 6

1.1 Internal registers and program units

Item	Purpose
DS1	Forward/reverse light step, 1 through 6
DS2	Pseudo-random table index, 0 through 11
DS3	Active mode memory: 0 invalid, 1 forward, 2 reverse, 3 random
DS30	HMI-owned mode request: 0 stop, 1 forward, 2 reverse, 3 random
C21 / C22 / C23	Effective requests after combining local pushbuttons and HMI
DS10 / DS11 / DS12	Mode-specific requested light numbers
DS13	Random lookup-table pointer (100 through 111)
DS20	Final selected light: 0 off, 1–6 light number
DS100--DS111	Random lookup table values
Main Program	Exclusive selection, mode entry, timer, request selection, simple output decode
I/O	Physical-to-intermediate and intermediate-to-physical mapping
Forward / Reverse / Random	State update plus mode-specific requested-light calculation

2 Program Architecture



CLICK project setup

Create four Subroutine Program pages named **IO**, **FORWARD**, **REVERSE**, and **RANDOM**. Each subroutine ends with **RETURN**; only Main ends with **END**. Conditional **CALL** rungs ensure that no sequence subroutine executes when the button selection is invalid. Keep each **C101--C106** command coil in exactly one Main-program rung; do not write those bits from any subroutine. The Random routine uses CLICK Single Copy pointer addressing: **DS[DS13]** means the DS register whose address number is stored in **DS13**. Configure Ethernet Port 1 as a Modbus TCP Server on TCP port 502. The HMI is a Modbus TCP client and writes only **DS30**; it should not write **DS3**, **DS20**, or **C101--C106**.

3 PC HMI Communication and Remote-Control Contract

The demonstration may be operated from a Windows PC running FUXA. The CLICK C0-11DRE-D Ethernet port is configured as a Modbus TCP server and FUXA is the Modbus TCP client. The tested connection for this project is shown below.

Setting	Configuration
CLICK Ethernet address	192.168.0.10
CLICK Modbus TCP server	Enabled on Port 1
TCP port	502
FUXA connection type	ModbusTCP, connection option TcpPort
FUXA socket reuse	Reuse
Typical polling interval	500 ms
HMI command register	DS30 / Modbus holding register 400030

FUXA mode-button write values

All four HMI buttons write the same PLC register. Only the constant value changes.

```
STOP -> write 0 to DS30
FORWARD -> write 1 to DS30
REVERSE -> write 2 to DS30
RANDOM -> write 3 to DS30
```

Variable ownership

DS30 is the HMI command variable: FUXA writes it and the PLC reads it. The PLC owns **DS3** (active mode), **DS1/DS2** (sequence state), **DS20** (final requested light), and the output command bits. This separation keeps troubleshooting straightforward and avoids two parts of the system fighting over the same runtime variable.

3.1 Recommended FUXA tags

FUXA tag	CLICK	Modbus type/address	Use
HMI_Mode	DS30	Holding Register 400030	Writable mode command, 0–3
ActiveMode	DS3	Holding Register 400003	Read-only active mode
SequencePosition	DS1	Holding Register 400001	Read-only forward/reverse step
RandomPosition	DS2	Holding Register 400002	Read-only random table index
RequestedLight	DS20	Holding Register 400020	Read-only requested light, 0–6
Light1--Light6	Y001--Y006	Coils; use CLICK Address Picker	Read-only lamp indication

For FUXA holding-register tags, use `Int16`; the tested FUXA address-offset entry for DS20 is 20 and displays as 400020. Use the same pattern for DS1, DS2, DS3, and DS30. For the physical outputs, create Boolean Coil tags for Y001--Y006; obtain their exact Modbus coil addresses from CLICK's Address Picker with Modbus addresses displayed.

Remote/local conflict behavior

The HMI is not given priority over the local buttons, and the local buttons are not given priority over the HMI. If they request different modes at the same time, more than one effective request becomes true. The existing exclusive-selection logic then sets all mode bits false and the machine fails off. If both sources request the same mode, that still counts as one effective request and operation continues normally.

4 Main Program Page

Main is the only program unit that arbitrates operator requests. Rung order is part of the design and should be preserved.

Rung 000 – Acquire physical inputs

The first I/O call updates the local input image. It also writes the previous output image once, but the final I/O call later in the scan overwrites it with the new image.

```
CALL IO
```

Rungs 001–003 – Combine local and HMI requests

Each effective request has one OUT coil. A local request and the matching HMI request are parallel paths to the same effective bit. Thus the PLC sees one common set of requests regardless of where the operator command originated.

```
(C1 OR DS30=1) --- OUT C21 FWD_REQUEST
(C2 OR DS30=2) --- OUT C22 REV_REQUEST
(C3 OR DS30=3) --- OUT C23 RND_REQUEST
```

Rungs 004–007 – Exclusive mode decode

The normally closed contacts for the other effective requests enforce exactly-one-mode selection. Conflicting local/HMI requests or multiple local buttons make every mode bit and VALID false.

```
C21 /C22 /C23 --- OUT C11 FWD_OK
/C21 C22 /C23 --- OUT C12 REV_OK
/C21 /C22 C23 --- OUT C13 RND_OK
(C11 OR C12 OR C13) --- OUT C14 VALID
```

Rungs 008–014 – Mode entry and invalid reset

DS3 makes mode changes robust even if the operator changes directly from one single-button request to another without an intervening empty scan.

```
C11 DS3<>1 --- COPY 1 to DS1 C11 DS3<>1 --- COPY 1 to DS3
C12 DS3<>2 --- COPY 6 to DS1 C12 DS3<>2 --- COPY 2 to DS3
C13 DS3<>3 --- COPY 0 to DS2 C13 DS3<>3 --- COPY 3 to DS3
/C14 --- COPY 0 to DS3
```

Rungs 015–026 – First-scan Random lookup-table initialization

SC2 is ON only during the first PLC scan. These rungs load the fixed 12-step Random light table once when the PLC enters RUN. The table contains light numbers, not output bits.

```
SC2 --- COPY 4 to DS100 SC2 --- COPY 1 to DS101
SC2 --- COPY 6 to DS102 SC2 --- COPY 2 to DS103
SC2 --- COPY 5 to DS104 SC2 --- COPY 3 to DS105
SC2 --- COPY 1 to DS106 SC2 --- COPY 5 to DS107
SC2 --- COPY 2 to DS108 SC2 --- COPY 6 to DS109
SC2 --- COPY 4 to DS110 SC2 --- COPY 3 to DS111
```

Rung 027 – Common one-scan pace pulse

The timer's own normally closed done contact makes the done bit a one-scan pulse.

```
C14 /T1 --- TMR T1, 0.75 second, ON-delay, automatic reset
```

Rungs 028–030 – Conditional mode calls

Exactly one mode subroutine can execute for a valid button selection. Each routine updates its state and publishes one requested light number.

```
C11 --- CALL FORWARD ; requested light -> DS10  
C12 --- CALL REVERSE ; requested light -> DS11  
C13 --- CALL RANDOM ; requested light -> DS12
```

Rungs 031–034 – Select the final requested light

Main acts as a small multiplexer. Only one of C11--C13 can be true, so only one valid-mode COPY executes. An invalid selection explicitly requests 0 (all lights off).

```
C11 --- COPY DS10 to DS20  
C12 --- COPY DS11 to DS20  
C13 --- COPY DS12 to DS20  
/C14 --- COPY 0 to DS20
```

Debugging advantage

DS20 is the single value to watch when troubleshooting the light command: 0 means no light, 1 means Light 1, through 6 for Light 6. The mode-specific details stay inside the mode subroutines instead of appearing as large Boolean expressions on every output rung.

Rungs 035–042 – Simple output decode, apply outputs, END

Each intermediate output bit has exactly one OUT instruction in the entire project. The output section now contains only six simple comparisons.

```
DS20=1 --- OUT C101
DS20=2 --- OUT C102
DS20=3 --- OUT C103
DS20=4 --- OUT C104
DS20=5 --- OUT C105
DS20=6 --- OUT C106
CALL IO
END
```

5 I/O Subroutine Program Page

This page is the only place where physical X and Y addresses appear. Sequence routines use only intermediate variables.

IO Rungs 000–002 – Input image

```
X001 --- OUT C1 IN_FORWARD
X002 --- OUT C2 IN_REVERSE
X003 --- OUT C3 IN_RANDOM
```

IO Rungs 003–008 – Physical output map

Each physical output has exactly one OUT instruction in the project.

```
C101 --- OUT Y001 LIGHT1
C102 --- OUT Y002 LIGHT2
C103 --- OUT Y003 LIGHT3
C104 --- OUT Y004 LIGHT4
C105 --- OUT Y005 LIGHT5
C106 --- OUT Y006 LIGHT6
```

IO Rung 009 – Return

```
RETURN
```

Why I/O is called twice

The first call gives Main the current physical input states. After the selected mode subroutine publishes its requested light, Main selects that value into DS20 and performs the simple one-of-six output decode. The second I/O call transfers C101--C106 to Y001--Y006. The second write is the controlling write for the scan, preserving a clean I/O page without adding a one-scan output delay.

6 Effective-Request Interlock Truth Table

The exclusive decoder acts on C21--C23, not directly on the physical inputs. This one table therefore covers local operation, HMI operation, and mixed operation.

C21	C22	C23	C11	C12	C13
0	0	0	0	0	0
1	0	0	1	0	0
0	1	0	0	1	0
0	0	1	0	0	1
1	1	0	0	0	0
1	0	1	0	0	0
0	1	1	0	0	0
1	1	1	0	0	0

Fail-off behavior

For every conflicting-request row, C11--C13 are all false and Main skips all three mode calls. The normally closed /C14 rung copies 0 to DS20; therefore all six simple output-decode comparisons are false and C101--C106 are written off. The final I/O call commands Y001--Y006 off in that same PLC scan.

7 Forward Subroutine Program Page

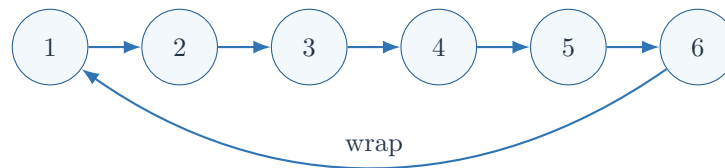
Forward advances DS1 from 1 through 6, wraps back to 1, and then publishes that light number in DS10. It does not write any output bit.

FORWARD Rungs 000–002 – Advance, wrap, publish request

```
T1 --- MATH DS1 = DS1 + 1
DS1 > 6 --- COPY 1 to DS1
SC1 --- COPY DS1 to DS10
```

FORWARD Rung 003 – Return

```
RETURN
```



8 Forward Verification

Hold only X001. Confirm DS3=1; DS1 begins at 1 and advances once per T1 pulse. Confirm DS10=DS1 and Main copies DS10 to DS20. The visible order is Y001,Y002,Y003,Y004,Y005,Y006, then repeat.

Single-scan protection and single writer

The increment-and-wrap method avoids same-scan cascading. When DS1=6 and T1 pulses, MATH produces 7 and the following wrap rung immediately copies 1 to DS1. The next rung publishes 1 in DS10; Main then selects it into DS20, so Y001 is not skipped. No Forward rung writes any C101--C106 bit.

9 Reverse Subroutine Program Page

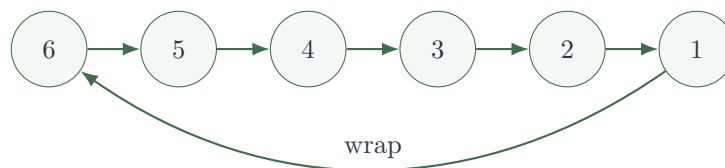
Reverse advances DS1 from 6 down to 1, wraps back to 6, and publishes that light number in DS11. It does not write any output bit.

REVERSE Rungs 000–002 – Advance, wrap, publish request

```
T1 --- MATH DS1 = DS1 - 1
DS1 < 1 --- COPY 6 to DS1
SC1 --- COPY DS1 to DS11
```

REVERSE Rung 003 – Return

```
RETURN
```



10 Reverse Verification

Hold only X002. Confirm DS3=2; DS1 begins at 6 and advances once per T1 pulse. Confirm DS11=DS1 and Main copies DS11 to DS20. The visible order is Y006,Y005,Y004,Y003,Y002,Y001, then repeat.

11 Random Subroutine Program Page

Random advances a repeatable irregular 12-step index in DS2. Instead of six output formulas, it converts the table index into a single requested light number using CLICK pointer addressing. The requested light is published in DS12.

RANDOM Rungs 000–003 – Advance, wrap, lookup, publish

```
T1 --- MATH DS2 = DS2 + 1
DS2 > 11 --- COPY 0 to DS2
SC1 --- MATH DS13 = DS2 + 100
SC1 --- COPY DS[DS13] to DS12
```

RANDOM Rung 004 – Return

```
RETURN
```

How the lookup works

The first-scan rungs load DS100--DS111 with the 12 light numbers. During Random, DS13=DS2+100; therefore DS[DS13] points at the matching table register. For example, DS2=3 makes DS13=103, so DS[DS13] means DS103, whose value is 2. The Single Copy places that value into DS12. Main then selects DS12 into DS20.

DS2	0	1	2	3	4	5	6	7	8	9	10	11
Table DS	100	101	102	103	104	105	106	107	108	109	110	111
Light	4	1	6	2	5	3	1	5	2	6	4	3

12 Verification Against Test Criteria

Test action	Expected result
No local buttons, DS30=0	C14=0; all intermediate and physical outputs off.
Hold only X001	Forward sequence begins at Y001, steps every 0.75 seconds, and wraps.
Hold only X002	Reverse sequence begins at Y006, steps every 0.75 seconds, and wraps.
Hold only X003	DS12 follows the 12-step lookup table, DS20=DS12, and the irregular light sequence repeats continuously.
FUXA writes DS30=1	With no conflicting local request, Forward becomes active exactly as if X001 were selected.
FUXA writes DS30=2	With no conflicting local request, Reverse becomes active.
FUXA writes DS30=3	With no conflicting local request, Random becomes active.
FUXA writes DS30=0	With no local button held, all modes become invalid and all lights turn off.
Local Forward + HMI Reverse	C21=1 and C22=1; all mode bits are false and outputs fail off.
Press any two buttons	All mode bits false; /C14 copies 0 to DS20; all lights turn off in the same scan.
Press all three buttons	Same fail-off result as a two-button combination.
Change directly between valid modes	DS3 detects the change and initializes the new mode correctly.

13 HMI Verification

Before relying on remote control, verify the communication path in stages:

1. Confirm FUXA reports the CLICK_PLC Modbus TCP connection as `Ok`.
2. Read DS20 as FUXA tag `RequestedLight`; its value should follow the running sequence.
3. Create writable tag `HMI_Mode` for DS30 and verify that writing 0, 1, 2, and 3 changes the PLC register accordingly.
4. Verify C21--C23 follow the combined local/HMI request logic and that DS3 reports the resulting active mode.
5. Add Boolean Coil tags for Y001--Y006 and bind them to six FUXA LED indicators so the HMI displays the actual PLC output image.
6. Test a deliberate conflict, such as local Forward with HMI Reverse; all lights should turn off.

14 Implementation Checks

1. Create all four subroutine program pages and end each with `RETURN`.
2. Configure CLICK Ethernet Port 1 as a Modbus TCP Server on TCP port 502 and verify the HMI connection before enabling remote control.
3. Reserve DS30 as the HMI-owned mode request: 0 Stop, 1 Forward, 2 Reverse, 3 Random. The PLC reads DS30 but does not continuously write it.
4. Build C21--C23 as effective requests using local input image OR matching DS30 compare conditions, then perform the exclusive decode from C21--C23.
5. Keep exactly one physical OUT instruction for each Y001--Y006, all on I/O.
6. Keep exactly one OUT instruction for each C101--C106, all in Main's simple DS20 output-decode section. Do not use `SET`, `RESET`, or additional OUT instructions on those bits elsewhere.
7. Use DS10, DS11, and DS12 as the Forward, Reverse, and Random requested-light registers, and DS20 as the final selected request.
8. Initialize DS100--DS111 on SC2 (first scan) with the documented Random lookup values.
9. In Random, calculate $DS13 = DS2 + 100$ and use Single Copy pointer addressing `DS[DS13]` as the source for DS12. Pointer addressing is supported by CLICK Single Copy.
10. Implement Forward as $DS1 = DS1 + 1$ with a $DS1 > 6$ wrap to 1, and Reverse as $DS1 = DS1 - 1$ with a $DS1 < 1$ wrap to 6.
11. Confirm the CLICK timer preset equals 0.75 seconds, then test all four invalid combinations: the three button pairs and all three buttons.

Recommended online watch list

During commissioning, monitor DS30, C21--C23, C11--C14, DS1--DS3, DS10--DS13, DS20, and C101--C106. In particular, DS20 should always tell you directly which light Main intends to energize.